One C++26 App, Three Ways

By Hand, By AI, and Together

Mike Spertus | mike@spertus.com

GlobalCpp | March 7, 2026

An Inflection Point

> *I rapidly went from about 80% manual+autocomplete coding and 20% agents in November to 80% agent coding and 20% edits+touchups in December... This is easily the biggest change to my basic coding workflow in ~2 decades of programming and it happened over the course of a few weeks.*

Andrej Karpathy, Co-founder of OpenAI (coiner of "Vibe Coding")

Agenda: What Does This Mean?

1

Build

The same application three ways

By hand | Vibe coding | Collaborating with AI

2

Compare

The code, the experience, and the outcome

Code quality | Development speed | Maintainability

3

Learn

How to get the most from today's tools

Best practices for AI-assisted C++ development

But What App?

Here is what ChatGPT says they are best at (and indeed we use them for this at my work)

AI coding tools like **Replit** tend to work best when the application has **clear structure, strong libraries, and limited system complexity**. Typical sweet spots:

1. CRUD / database-backed web apps

Examples:

- dashboards
- admin tools
- internal business tools
- simple SaaS products

Why they work well:

- repetitive patterns (routes, models, forms)
- lots of existing examples
- frameworks guide structure.

But What About Apps That...

- Don't have well-understood patterns
- Don't have lots of existing examples
- Aren't based on frameworks

?

Let's Do a Specialized C++26 App

- ✓ Patterns not understood yet
- ✓ Not many examples
- ✓ Not based on a framework

Sample Program

I have created a sample program that does XML data binding for C++26 using many advanced C++ features.

I'll begin by describing the program.

XML Schema

An XML Schema specifies what an XML document can look like by defining a series of types.

```
<xs:schema targetNamespace=http://tempuri.org/XMLSchema.xsd ...>
  <xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="to" type="xs:string"/>
        <xs:element name="from" type="xs:string"/>
        <xs:element name="heading" type="xs:string"/>
        <xs:element name="body" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Document Matching That Schema

```
<note>
  <from>Mike</from>
  <to>Students</to>
  <heading>XML Data Binding Program</heading>
  <body>
    Lorem ipsum dolor sit amet, consectetur
    adipiscing elit. Quisque fermentum augue in sem
    suscipit, et posuere ligula sollicitudin. Nullam
    venenatis ex ac bibendum egestas.
  </body>
</note>
```

Data Binding

Generate C++ classes that match the types in an XML schema:

```
struct note_type {
    std::string to;
    std::string from;
    std::string heading;
    std::string body;
};
```

As well as functions to serialize and deserialize to XML.

This makes working with XML data easy and natural in C++.

Philosophy

Design patterns, metaprogramming, and code generation

Programming with Design Patterns

The Gang of Four's Design Patterns: Elements of Reusable Object-Oriented Software

- ACM SIGPLAN Programming Language Achievement Award
- One of only three software engineering books on Wikipedia's list of Important Publications in CS

A design pattern is a general reusable solution to a commonly occurring problem... not a finished design that can be transformed directly into source code.

Hmm, are we sure? What if we could generate code directly from our designs?

Andrei Alexandrescu: Modern C++ Design

Major theme: Implementing Design Patterns as Templates

John Vlissides (Gang of Four) writes in the foreword:

"Generic components implement good designs in easy-to-use, mixable-and-matchable templates. They do pretty much what code generators do: produce boilerplate code for compiler consumption. The difference is that they do it within C++, not apart from it. The result is seamless integration with application code."

Our Program Will Use Modern C++ to Automate Design Patterns

1. Factories
2. Visitors
3. Parallel Hierarchies

Powered by Modern C++ (Really Is Better)

C++17	deduced non-type template parameters, fold expressions, inline variables, variant, optional, string_view
C++20	concepts, ranges, std::format
C++23	formatting ranges, inheriting from std::variant
C++26	reflection, std::optional<T&>

Factories

The AbstractFactory and ConcreteFactory design patterns

Example: AbstractFactory & ConcreteFactory

These are two of the most popular Design Patterns.

In my experience, most large scale programs have something like the following...

AbstractFactory Pattern

```
class WidgetFactory {
public:
  virtual unique_ptr<Window>
    CreateWindow() = 0;
  virtual unique_ptr<Button>
    CreateButton() = 0;
  virtual unique_ptr<ScrollBar>
    CreateScrollBar() = 0;
};
```

ConcreteFactory Pattern

By using a factory for your particular UI, you don't need to worry about creating incompatible objects, like accidentally creating a Windows widget in an Android app.

```
class MSWindowsWidgetFactory
  : public WidgetFactory {
  unique_ptr<Window> CreateWindow() override
    { return make_unique<MSWindow>(); }
  /* ... */
};

class AndroidWidgetFactory
  : public WidgetFactory { ... };
```

Putting Them Together

Easy-to-use and hard to misuse:

```
unique_ptr<WidgetFactory> widgets
  = make_unique<MSWindowsWidgetFactory>();

// All code below here can only use
// generic widget functionality so
// if we switch to an
// AndroidWidgetFactory later, it will
// still work

auto myButton = widgets.CreateButton();
```

So What's the Problem?

Factories are easy to use but hard to write.

- There are actually dozens of different kinds of widget
- Manually implementing all of the different factories is:
  - tedious: programmers will avoid using factories even when beneficial
  - error-prone and brittle: vulnerable to cut-and-paste errors, getting out of sync

Can we use a template to generate for us?

Techniques We Will Need

1. Typelists
2. Simulating template virtuals
3. Parallel hierarchies
4. Reflection

Typelists

Sometimes we just want a list of types — not to create values of those types but just to power variadic templates. For example, feeding the list of types made by a factory into the factory template.

Code is beautifully simple:

```
template<typename ...Ts> struct typelist {};
```

Manipulate Typelists with Metafunctions

```
template<typename T, typename A, typename B> struct Replace;

template<typename A, typename B>
struct Replace<typelist<>, A, B> {
    using type = typelist<>;
};

template<typename H, typename... Ts, typename A, typename B>
struct Replace<typelist<H, Ts...>, A, B>
  : public Append<typelist<H>,
      typename Replace<typelist<Ts...>, A, B>::type> {};

template<typename... Ts, typename A, typename B>
struct Replace<typelist<A, Ts...>, A, B> {
    using type = typelist<B, Ts...>;
};

static_assert(is_same_v<
    Replace_t<typelist<int, char, vector>, vector, list>,
    typelist<int, char, list>>);
```

Simulating Template Virtuals

Our factory pattern relies heavily on virtual methods.

However, method templates cannot be virtual. (Why?)

If we want to implement this design pattern as a template, we will need a solution.

Let's see if we can come up with a way to simulate virtual method templates.

What Would Template Virtuals Look Like?

The reason method templates can't be virtual is because they aren't methods — what would the vtable even look like?

However, we can call ordinary methods from our method templates.

So we will specify which specializations should call which method.

What Interface Do We Want to Support?

Suppose we have a method template. Now suppose we want f<int>, f<double>, and f<A> to be "virtuals" that can be overridden in a derived class.

```
struct S {
  template<typename T> void f();
};
```

First Swing

Works for int and double, but A may be abstract or lack a default constructor.

```
struct S {
  template<class T>
  void f() { return fHelper(T()); }

  virtual void fHelper(int &&) = 0;
  virtual void fHelper(double &&) = 0;
  virtual void fHelper(A &&) = 0;
};
```

Unfortunately, fHelper(A()) won't compile if A is abstract.

TT: The Tag Type

We need a "tag type" that is always: not abstract, default constructible, and cheap to construct.

If I construct with TT<A>{}, it's legal because it is not abstract and is default constructible, and it's cheap because it's just an empty class.

```
template<class T>
struct TT {};
```

Better

```
struct S {
  template<class T>
  void f() { return fHelper(TT<T>()); }

  virtual void fHelper(TT<int> &&) = 0;
  virtual void fHelper(TT<double> &&) = 0;
  virtual void fHelper(TT<A> &&) = 0;
};
```

A class that inherits from S can override the fHelper() methods.

Can We Automate?

The only problem is that it is tedious and error-prone to manually define all of the helper virtual methods.

Sounds like a job for variadics!

Let's begin by defining a class template that holds just one Helper, and then use variadics to inherit from all of them.

The Solution

```
template<typename T>
struct Holder {
  virtual void fHelper(TT<T> &&) = 0;
};

template<typename ...Ts>
struct ST : public Holder<Ts>... {
  template<class T>
  void f() { return fHelper(TT<T>()); }
};

using S = ST<int, double, A>;
```

S has virtual fHelper methods for int, double, and A.

Our Goal: Automate with Factory Templates

Easy to create:

```
using WidgetFactory
  = AbstractFactory<
      Window, Button, Scrollbar>;
using QtFactory
  = ConcreteFactory<WidgetFactory,
      QtWindow, QtButton, QtScrollbar>;
using GTKFactory
  = ConcreteFactory<WidgetFactory,
      GTKWindow, GTKButton, GTKScrollbar>;
```

Easy to use:

```
unique_ptr<WidgetFactory> wf
  = make_unique<QtFactory>();

// Easy to use
auto b = wf->Create<Button>();
```

Simulating "Template Virtuals" in Our Factories

One of the big problems is that factories use virtual methods so the abstract factory methods can be overridden, but method templates can't be virtual.

Let's apply the technique from a few slides ago for simulating template virtuals.

The Abstract Creator

Build up from a single doCreate. Since all of the doCreate methods have the same name and method hiding takes place, we need to be careful.

```
template<typename T>
struct abstract_creator {
  virtual unique_ptr<T>
    doCreate(TT<T>&&) = 0;
};
```

Abstract Factory (factory.h)

By using a variadic expansion of the base class, we inherit from all abstract creators at once.

```
template<typename ...Ts>
struct AbstractFactory
  : public abstract_creator<Ts>... {

  using abstract_creator<Ts>::doCreate...;

  template<typename T>
  unique_ptr<T> Create() {
    return doCreate(TT<T>{});
  }
};
```

Concrete Factory (factory.h)

By using a variadic expansion, we can generate all concrete factory methods automatically.

```
template<typename AbstractFact, typename ...Ts>
struct ConcreteFactory : public AbstractFact {

  using AbstractFact::doCreate;

  template<typename T>
  unique_ptr<typename T::base_type>
  doCreate(TT<typename T::base_type>&&) override {
    return make_unique<T>();
  }
};
```

C++26 Reflection

C++26 has an awesome reflection capability.

See Barry Revzin's recent GlobalCPP talk to come up to speed in just an hour.

One pain point: maintaining the list of classes in our factory. They are just the ones that inherit from, say, Widget.

Unfortunately, reflection doesn't directly give us all types that derive from a given type.

Can We Workaround?

We can enumerate all of the symbols in a namespace to get the result we want.

```
template<class Base>
using deriveds_of = [:
    substitute(
        ^^typelist,
        members_of(^^mpcs, unchecked)
          | std::views::filter(
              safe_is_base_of<Base>)
    )
:];
```

Visitors

Adding virtuals from outside the class

The Visitor Pattern

It is common to wish that a hierarchy of classes you are using had some extra virtuals. But you may not be able to add them:

- Maybe they're not your classes
- Maybe the virtuals you want only apply to your particular program
- It's not worth cluttering up a general class with program-specific virtuals

Suppose I Am Writing a Game

I am using a 3rd-party Animal hierarchy.

I wish it had a virtual method for hit points — but that is specific to my game.

What if there were a way for clients to "inject" virtuals into an external class hierarchy?

The Visitor Pattern

Specify behavior in a "visitor" class, then add an accept method to each class.

```
struct AnimalVisitor {
  virtual void visit(Gorilla &) const = 0;
  virtual void visit(Animal &) const = 0;
};

struct Animal {
  virtual void accept(AnimalVisitor const &v)
    { v.visit(*this); }
  ...
};

struct Gorilla {
  virtual void accept(AnimalVisitor const &v)
    { v.visit(*this); }
  ...
};
```

Using Visitors with Accept

```
struct HitPointVisitor : public AnimalVisitor {
  void visit(Gorilla&) const override
    { cout << 15; }
  void visit(Animal&) const override
    { cout << 2; }
};

unique_ptr<B> bp = make_unique<D>();
bp->accept(Printer()); // Prints "D"
```

Automating Visitors

I advise my students to make most of their hierarchies acceptable.

But putting in the accept method manually is cumbersome and error-prone copy pasta.

C++23 deducing this doesn't work with virtuals, so we can't just propagate from the base.

Our program will have a wrapper acceptable template that contains our classes and an accept method. Principled use of metaprogramming will ensure this template is always used.

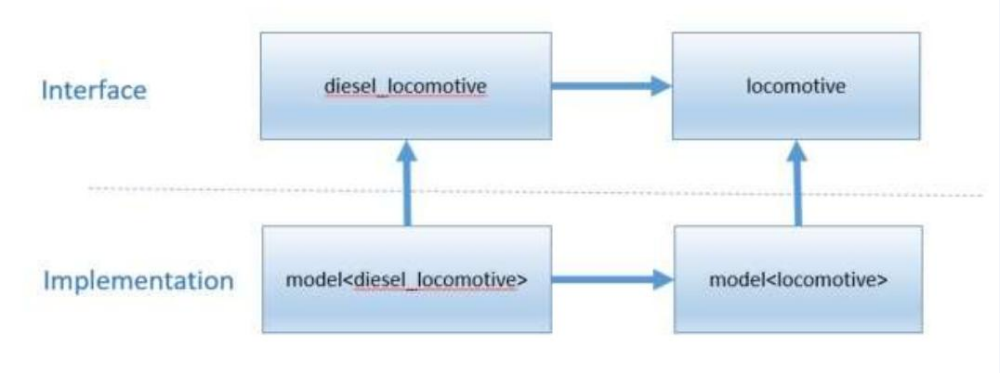
Parallel Hierarchies

Mirroring inheritance relationships across factory families

Parallel Hierarchies

Abstract classes for the abstract factory, concrete classes implementing them — they should have same inheritance relationships in a "parallel hierarchy".

For example, model trains implementing a train hierarchy.



The Missing Type Trait: bases_of

Unfortunately, there is no bases_of type trait. We've been trying since 2009 (N2965). With reflection, we don't need to wait!

```
template<class X>
using direct_bases_t = [:
    substitute(^^typelist,
        meta::bases_of(^^X, unchecked)
          | std::views::transform(
              [](auto b) {
                return meta::type_of(b);
              }))
:];
```

Represent Concrete Families as a Template

```
template<typename T>
struct model
  : virtual public parallel_bases<model, T, all_cars>,
    virtual public T {};

template<>
struct model<locomotive>
  : virtual public parallel_bases<
      model, locomotive, all_cars> {
  virtual void turnOnEngines() {
    cout << "Turned on electricity for train set\n";
  }
};

template<>
struct model<steam_locomotive>
  : virtual public parallel_bases<
      model, steam_locomotive, all_cars> {};
```

Our Sample Program By Hand

Putting it all together with XsdToCpp

XsdToCpp

We now have the tools we need for a principled implementation that is robust, extensible, and maintainable.

Shhh… I didn’t write it entirely by hand. I used AI for autocomplete and to explain C++ error messages

It was hit-or-miss with both of therse

But you could say the same about me!

Step 1: Read in the Schema

We use libstudxml to read in the schema and invoke a factory that constructs a hierarchy of types.

This gives us a faithful internal representation of the XML schema. But there are many things we might do with this:

- Generate declarations
- Generate definitions
- Generate structs / classes
- Convert to JSON schema
- Etc.

So we don't know what methods to put in...

Step 2: Making It Acceptable

Sounds like a job for accept!

In fact, that will be the only method in our type hierarchy.

Step 3: Making Formatters

We will have an abstract formatter class template that says how to format each XML schema type.

And concrete hierarchies for all the different types of things we'd like to generate. (This also leverages an IndentStream class that is handy for code generation by using a custom streambuf to autoindent each line)

This makes generation fully pluggable and extensible.

"Prefer concentrated complexity to diffuse complexity"

Demo

Vibe Coding

"Let's just say what we want and let Claude Code do it"

The Vibe Coding Prompt

```
Write a modern C++26 XML data binding program that includes enough of XML
schema to run the sample test correctly. Use libstudxml (available in
Conan for XML parsing). Make sure you follow modern C++ coding and design
best practices. Do not look at anything outside the project folder.

claude --dangerously-skip-permissions
```

Give it a test case and the prompt above.

It Did It in 1 Hour!

1hr

AI vibe coding

vs

Days

By hand

Not bad.

Adding Features

I just asked it for various features:

- Pluggable generation: classes with getters and setters
- More coverage of XML Schema
- Generate a JSON Schema from an XML Schema
- Etc.

Also done successfully in under an hour!

The manual version would take me a week. But OMG!

Is There a Catch?

The code is solid tactical C++ code, but it lacks abstraction.

The complexity is diffuse rather than concentrated — unlike our manual example.

We can see this by looking at the commits, where the blast radius for each new feature is scattered throughout the program.

Note: This is not seen in CRUD web apps, because AI has architectural patterns to follow. Most of the best practices around using Claude Code are about avoiding diffuse complexity.

Where Do We Stand?

Due to limited adherence to architectural principles, it would be difficult for a human to maintain and review this program.

Worse yet, Karpathy and Cherny (inventor of Claude Code) say production code still needs human review due to subtle mistakes.

> *"Claude Code wiped our production database with a Terraform command."*

Note: One day this may not matter once AI becomes a "compiler". No one cares if the machine code generated by a compiler follows the SOLID principles.

Is There a "Goldilocks Solution"?

Building Together

Human architecture + AI implementation

How to Work Together

1	**Minimal seed**	Create a minimal version using the full architecture (just the note example)
2	**CLAUDE.md**	Give it design principles, including to model off the existing code
3	**Plan mode**	Use plan mode for each feature
4	**Review**	Review the feature after Claude implements it
5	**Simplify**	Regularly run the /simplify command

This Worked Great!

In a couple of hours, we implemented a boatload of features with full test suites.

Including complex ones like XML schema extension types and translation to JSON Schema — either of which would have individually taken me over a couple of hours by hand.

I occasionally found important problems (both strategic and tactical) in the plan and the review, but that felt similar to being a team lead.

Let's take a look.

Even Better

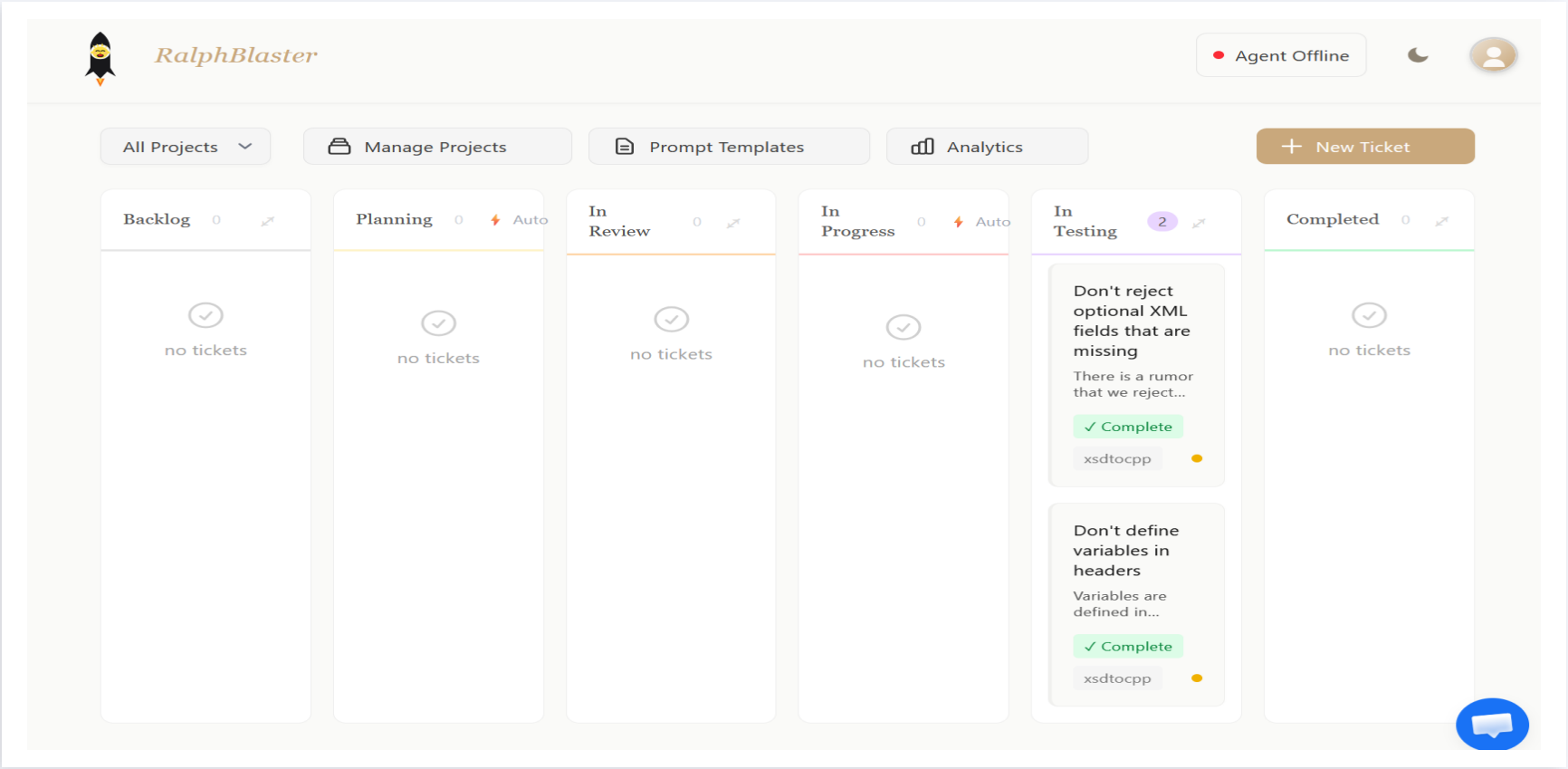
While this took a few hours, almost all of it was Claude working in the background.

Rather than using the Claude terminal or VSCode extension, you can use tools like Ralphblaster that give Claude Code a Kanban board.

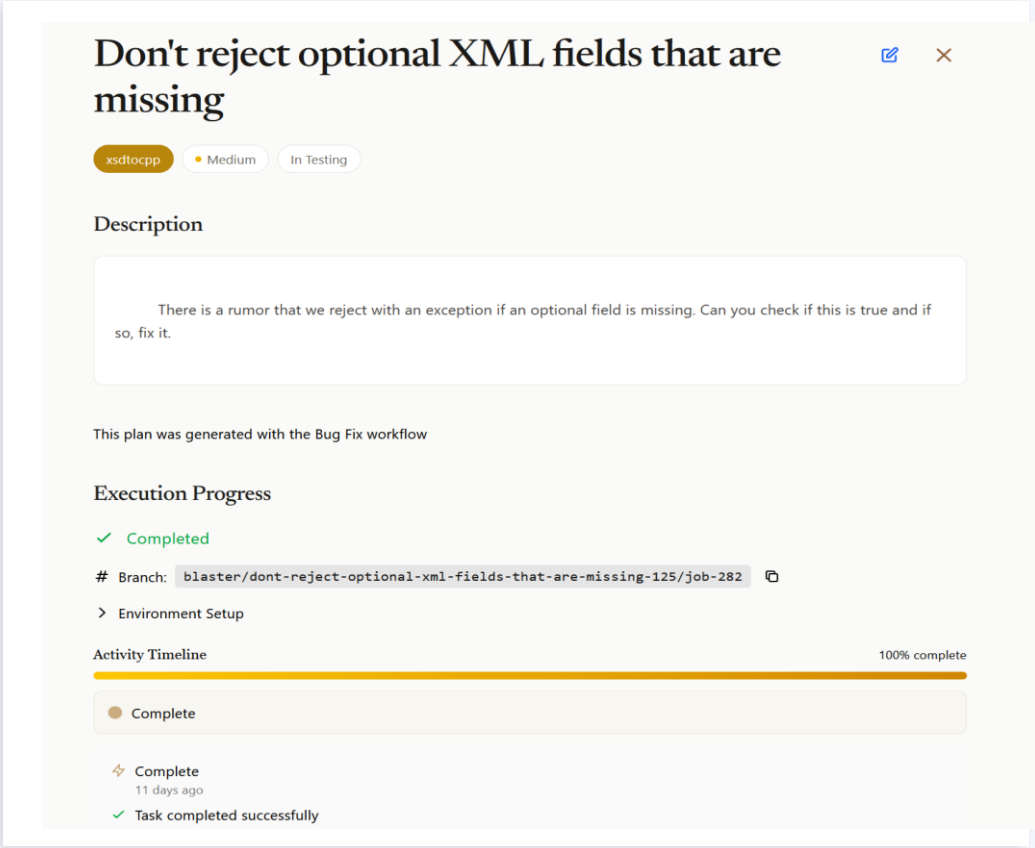
The task gets assigned to you when there is something for you to do.

Now it only takes a few minutes of my time.

RalphBlaster: AI-Powered Kanban



RalphBlaster: Task Detail



Summary

Key takeaways from building one app three ways

AI is the biggest advance I've seen in my 58 years of programming

You aren't competitive without AI

But AI is not competitive without you

I hope this talk gave you some ideas for how to get the most from it.

Questions?

Mike Spertus | mike@spertus.com